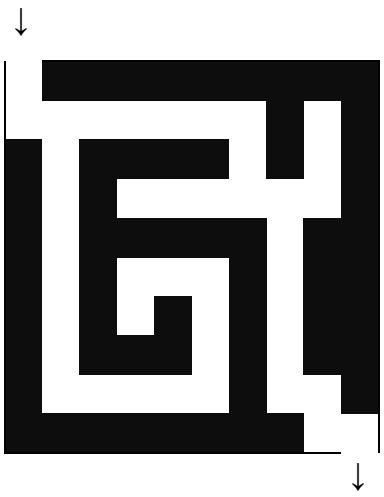
Exercice 1 : *Thèmes abordés : programmation Python, tuples et listes*

L'objectif de cet exercice est de mettre en place une modélisation d'un jeu de labyrinthe en langage Python.

On décide de représenter un labyrinthe par un tableau carré de taille `n`, dans lequel les cases seront des 0 si l'on peut s'y déplacer et des 1 s'il s'agit d'un mur. Voici un exemple de représentation d'un labyrinthe :



```
laby=[[0,1,1,1,1,1,1,1,1,1],
      [0,0,0,0,0,0,0,1,0,1],
      [1,0,1,1,1,1,0,1,0,1],
      [1,0,1,0,0,0,0,0,0,1],
      [1,0,1,1,1,1,1,0,1,1],
      [1,0,1,0,0,0,1,0,1,1],
      [1,0,1,0,1,0,1,0,1,1],
      [1,0,1,1,1,0,1,0,1,1],
      [1,0,0,0,0,0,1,0,0,1],
      [1,1,1,1,1,1,1,1,0,0]]
```

L'entrée du labyrinthe se situe à la première case du tableau (celle en haut à gauche) et la sortie du labyrinthe se trouve à la dernière case (celle en bas à droite).

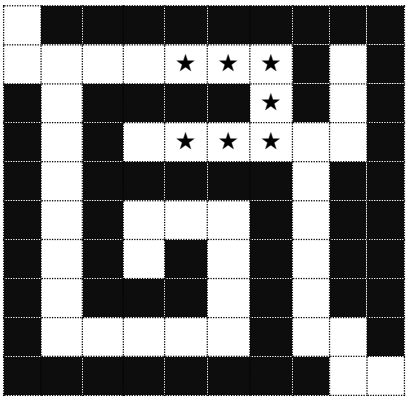
1. **Proposer**, en langage Python, une fonction `mur`, prenant en paramètre un tableau représentant un labyrinthe et deux entiers `i` et `j` compris entre 0 et `n-1` et qui renvoie un booléen indiquant la présence ou non d'un mur. Par exemple :

```
>>mur(laby, 2, 3)
True
>>mur(laby, 1, 8)
False
```

Un parcours dans le labyrinthe va être représenté par une liste de **cases.** Il s'agit de couples `(i,j)` où `i` et `j` correspondent respectivement aux numéros de ligne et de colonne des cases successivement visitées au long du parcours. Ainsi, la liste suivante

`[(1,4),(1,5),(1,6),(2,6),(3,6),(3,5),(3,4)]`

correspond au parcours repéré par des étoiles ⋆ ci-contre :



La liste `[(0,0),(1,0),(1,1),(5,1),(6,1)]` ne peut correspondre au parcours d'un labyrinthe car toutes les cases parcourues successivement ne sont pas adjacentes.

2. On considère la fonction `voisine` ci-dessous, écrite en langage Python, qui prend en paramètres deux cases données sous forme de couple.

```
def voisine(case1, case2) :
    l1, c1 = case1
    l2, c2 = case2
    # on vous rappelle que **2 signifie puissance 2
    d = (l1-l2)**2 + (c1-c2)**2
    return (d == 1)
```

2.a. Après avoir remarqué que les quantités `l1-l2` et `c1-c2` sont des entiers, **expliquer** pourquoi la fonction `voisine` indique si deux cases données sous forme de tuples `(l,c)` sont adjacentes.

2.b. **En déduire** une fonction `adjacentes` qui reçoit une liste de cases et renvoie un booléen indiquant si la liste des cases forme une chaîne de cases adjacentes.

Un parcours sera qualifié de ***compatible avec le labyrinthe*** lorsqu'il s'agit d'une succession de cases adjacentes accessibles (non murées). On donne la fonction `teste(cases, laby)` qui indique si le chemin `cases` est un chemin possible compatible avec le labyrinthe `laby` :

```
def teste(cases, laby) :
    if not adjacentes(cases) :
        return False
    possible = True
    i = 0
    while i < len(cases) and possible:
        if mur(laby, cases[i][0], cases[i][1]) :
            possible = False
        i = i + 1
    return possible
```

3. **Justifier** que la boucle de la fonction précédente se termine.
4. **En déduire** une fonction `echappe(cases, laby)` qui indique par un booléen si le chemin `cases` permet d'aller de l'entrée à la sortie du labyrinthe `laby`.

EXERCICE 2 :

Principaux thèmes abordés : structure de données (tableaux, dictionnaires) et langages et programmation (spécification).

Objectif de l'exercice :

Les Aventuriers du Rail© est un jeu de société dans lequel les joueurs doivent construire des lignes de chemin de fer entre différentes villes d'un pays.

La carte des liaisons possibles dans la région Occitanie est donnée en **annexe 1 de l'exercice 2**. Dans l'**annexe 2 de l'exercice 2**, les liaisons possédées par le joueur 1 sont en noir, et celles du joueur 2 en blanc. Les liaisons en gris sont encore en jeu.

Codages des structures de données utilisées :

- **Liste des liaisons d'un joueur :** Toutes les liaisons directes (sans ville intermédiaire) construites par un joueur seront enregistrées dans une variable de type "**tableau de tableaux**".

  Le joueur 1 possède les lignes directes **"**Toulouse-Muret", "Toulouse-Montauban", "Gaillac-St Sulpice" et "Muret-Pamiers" (liaisons indiquées en noir dans l'**annexe 2 de l'exercice 2**). Ces liaisons sont mémorisées dans la variable ci-contre.

```
liaisonsJoueur1 = [
 ["Toulouse","Muret"],
 ["Toulouse","Montauban"],
 ["Gaillac","St Sulpice"],
 ["Muret","Pamiers"]
]
```

  Remarque : *Seules les liaisons directes existent, par exemple* ***["Toulouse","Muret"]*** *ou* ***["Muret","Toulouse"]****. Par contre, le tableau* ***["Toulouse","Mazamet"]*** *n'existe pas, puisque la ligne Toulouse-Mazamet passe par Castres.*

- **Dictionnaire associé à un joueur :** On code la liste des villes et des trajets possédée par un joueur en utilisant un **dictionnaire de tableaux**. Chaque **clef de ce dictionnaire** est une ville de départ, et chaque **valeur** est un **tableau** contenant les villes d'arrivée possibles en fonction des liaisons possédées par le joueur.

  Le **dictionnaire de tableaux** du joueur 1 est donné ci-contre :

```
DictJoueur1 = {
    "Toulouse":["Muret","Montauban"],
    "Montauban":["Toulouse"],
    "Gaillac":["St Sulpice"],
    "St Sulpice":["Gaillac"],
    "Muret":["Toulouse","Pamiers"],
    "Pamiers":["Muret"]}
```

1. Expliquer pourquoi la liste des liaisons suivante n'est pas valide :

```
tableauliaisons = [["Toulouse","Auch"], ["Luchon","Muret"],
["Quillan","Limoux"] ]
```

2. Cette question concerne le joueur n°2 *(Rappel : les liaisons possédées par le joueur n°2 sont représentées par un rectangle blanc dans l'**annexe 2 de l'exercice 2**)*.

a) Donner le tableau `liaisonsJoueur2`, des liaisons possédées par le joueur n°2.

b) Recopier et compléter le dictionnaire suivant, associé au joueur n°2 :

```
DictJoueur2 = {
    "Toulouse":["Castres","Castelnaudary"],
    …
}
```

3. À partir du tableau de tableaux contenant les liaisons d'un joueur, on souhaite construire le dictionnaire correspondant au joueur. Une première proposition a abouti à la fonction **`construireDict`** ci-dessous.

```
1   def construireDict(listeLiaisons):
2     """
3     listeLiaisons est un tableau de tableaux représentant la
4     liste des liaisons d'un joueur comme décrit dans le problème
5     """
6     Dict={}
7     for liaison in listeLiaisons :
8       villeA = liaison[0]
9       villeB = liaison[1]
10      if not villeA in Dict.keys() :
11        Dict[villeA]=[villeB]
12      else :
13        destinationsA = Dict[villeA]
14        if not villeB in destinationsA :
15          destinationsA.append(villeB)
16    return Dict
```

a) Écrire sur votre copie un assert dans la fonction **`construireDict`** qui permet de vérifier que la **`listeLiaisons`** n'est pas vide.

b) Sur votre copie, donner le résultat de cette fonction ayant comme argument la variable **`liaisonsJoueur1`** donnée dans l'énoncé et expliquer en quoi cette fonction ne répond que partiellement à la demande.

c) La fonction `construireDict`, définie ci-dessus, est donc partiellement inexacte.

Compléter la fonction `construireDict` pour qu'elle génère bien l'ensemble du dictionnaire de tableaux correspondant à la liste de liaisons données en argument. À l'aide des numéros de lignes, on précisera où est inséré ce code.

EXERCICE 3 :

Principaux thèmes abordés : Traitement de données en tables (CSV) et langages et programmation (spécification).

Afin d'améliorer l'ergonomie d'un logiciel de traitement des inscriptions dans une université, un programmeur souhaite exploiter l'intelligence artificielle pour renseigner certains champs par auto-complétion. Il s'intéresse au descripteur « genre » (masculin, féminin ou indéterminé). Pour cela il propose d'exploiter les dernières lettres du prénom pour proposer automatiquement le genre.

Pour vérifier son hypothèse, il récupère un fichier CSV associant plus de 60 000 prénoms du monde entier au genre de la personne portant ce prénom. En utilisant seulement la dernière lettre, le taux de réussite de sa démarche est de 72,9% avec la fonction définie ci-dessous :

```
def genre(prenom):
    liste_M = ['f', 'd', 'c', 'b', 'o', 'n', 'm', 'l', 'k',
               'j', 'é', 'h', 'w', 'v', 'u', 't', 's', 'r',
               'q', 'p', 'i', 'þ', 'z', 'x', 'ç', 'ö', 'ã',
               'â', 'ï', 'g']
    liste_F = ['e', 'a', 'ä', 'ü', 'y', 'ë']

    if prenom[len(prenom)-1].lower() in liste_M :
        return "M"
    elif prenom[len(prenom)-1].lower() in liste_F :
        return "F"
    else :
        return "I"


# Pour rappel, C.lower() convertit le caractère C en minuscule.
```

1. Appropriation

a) Expliquer ce qu'est un fichier CSV.

b) Donner le type de l'argument **`prenom`** de la fonction **`genre`**, et le type de la réponse renvoyée.

2. Développement

Pour effectuer son étude sur les prénoms à partir du fichier CSV, le programmeur souhaite utiliser la bibliothèque csv.

a) La bibliothèque csv est un module natif du moteur python.

Donner, dans ce cas, l'instruction d'importation.

b) Au cours du développement de son projet, le programmeur souhaite insérer une assertion sur l'argument donné à la fonction.

Proposer une assertion sur le type de l'argument qui corrige une erreur lorsque le type ne correspond pas à celui attendu.

c) Avant le déploiement de sa solution, le programmeur décide de rendre sa fonction plus robuste.

Pour cela il veut remplacer l'assertion proposée dans la question **2.b)** par une gestion de l'argument pour éviter toutes erreurs empêchant la poursuite du programme.

Proposer alors une ou plusieurs instructions en Python utilisant l'argument afin de s'assurer que la fonction se termine quel que soit le type de l'argument.

3. Améliorations

En prenant en compte les deux dernières lettres du prénom, il parvient à augmenter son taux de réussite à 74,4%. Pour cela, son étude du fichier CSV lui permet de créer deux listes : **`liste_M2`** pour les terminaisons de deux lettres associées aux prénoms masculins et **`liste_F2`** pour les prénoms féminins.
Sur votre copie, recopier et modifier la structure conditionnelle (lignes 8 à 13) de la fonction **`genre`** afin de prendre en compte les terminaisons de deux lettres des listes **`liste_M2`** et **`liste_F2`**.

EXERCICE 4 :

Cet exercice est un exercice d'algorithmique et de programmation en langage Python utilisant un fichier csv.

Partie 1

Dans un fichier villes56.csv, on a regroupé les neuf villes du Morbihan de plus de 10 000 habitants en 2012.

dep	nom	nb_hab_1999	nb_hab_2012	densite	superficie	alt_min	alt_max
56	Lorient	59224	58100	3272	17.48	0	46
56	Vannes	51759	53000	1625	32.3	0	56
56	Lanester	21901	22500	1209	18.37	0	51
56	Ploemeur	18301	18200	448	39.72	-1	55
56	Hennebont	13410	14600	818	18.57	0	82
56	Pontivy	13501	13700	553	24.85	48	192
56	Auray	10899	12500	1783	6.91	0	43
56	Guidel	9155	10100	198	52.29	0	70
56	Saint-Avé	8298	10100	400	26.09	12	136

1) En appliquant le programme suivant :

Programme 1

```
1   import csv
2
3   def readCSV( filename):
4      '''
5      filename est une chaine de caractères correspondant au nom d'un fichier du type csv
6      La fonction renvoie une variable de type list
7      '''
8       with open(filename, mode = 'r', encoding='utf-8', newline = '') as csvFile:
9           fichier = csv.DictReader(csvFile)

10          table1 = [dict(ligne) for ligne in fichier]
11               # les lignes 9 et 10 permettent d'obtenir une liste de dictionnaires dont
    les clés correspondent à la première ligne du fichier csvFile
12      csvFile.close()
13      return table1
14
15  listeVilles = readCSV("villes56.csv")
16  print(listeVilles)
```

a) À partir des quatre propositions ci-dessous, **indiquer** ce que signifie « `utf-8` » à la ligne 8.
Réponse A : type de codage de caractères informatiques.
Réponse B : nom du dossier contenant le fichier.
Réponse C : nom du fichier.
Réponse D : taille du fichier.

b) **Indiquer** ce que signifie le « `mode ='r'` » à la ligne 8.

2) Après exécution du programme Programme 1, voici le début de l'affichage obtenu :

```
[{'dep': '56',
 'nom': 'Lorient',
 'nb_hab_1999': '59224',
 'nb_hab_2012': '58100',
 'densite': '3272',
 'superficie': '17.48',
 'alt_min': '0',
 'alt_max': '46'},
{'dep': '56',……
```

a) **Indiquer** le type de la variable `listeVilles`.

b) **Indiquer** ce que renvoie l'instruction `listeVilles`[1] en précisant son type.

c) **Écrire** une instruction permettant d'obtenir le nombre d'habitants à Pontivy en 2012.

Partie 2

On admet qu'après un traitement, les données figurant dans les colonnes 2 à 7 du tableau `listeVilles` sont des entiers ou des flottants. On considère que `listeVilles` résulte de l'exécution du programme Programme 1.

1) **Indiquer** ce que renvoie l'instruction suivante appliquée à `listeVilles` :
```
>>>[(d['nom'],d['alt_max']) for d in listeVilles if d['alt_max']>100]
```

2) **Écrire** en langage Python une fonction `variations` qui prend en paramètre `listeV`, du même type que la variable `listeVilles`, et renvoie la différence totale entre le nombre d'habitants en 2012 et le nombre d'habitants en 1999 cumulée pour l'ensemble des villes.

3) **Écrire** en langage Python une fonction `surmax` qui prend en paramètre `listeV`, du même type que la variable `listeVilles`, et renvoie la ville ayant la superficie la plus grande. On admet que deux villes n'ont pas la même superficie.

EXERCICE 5 :

Cet exercice porte sur les structures de données (listes, p-uplets et dictionnaires).

On dispose de la liste `jours` suivante et du dictionnaire `mois` suivant :

```
jours=["dimanche", "lundi", "mardi", "mercredi", "jeudi",
"vendredi", "samedi"]

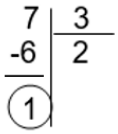
mois={1 :("janvier",31) , 2 :("février",28) , 3 :("mars",31),
4 :("avril",30) , 5 :("mai",31) , 6 :("juin",30) ,
7 :("juillet",31) , 8 :("aout",31) , 9 :("septembre",30) ,
10 :("octobre",31) , 11 :("novembre",30) ,
12 :("décembre",31)}
```

1.

a. A partir de la liste `jours`, comment obtenir l'élément `"lundi"` ?

b. On rappelle que l'opérateur % (« modulo ») renvoie le reste de la division entière (division euclidienne).

Exemple : 7%3 renvoie 1 qui est le reste de la division de 7 par 3 :



Que renvoie l'instruction `jours[18%7]` ?

2. On rappelle que `jours.index[element]` renvoie l'indice de `element` dans la liste `jours` par exemple `jours.index["mercredi"]` renvoie 3. Le nom du jour actuel est stocké dans une variable `j` (par exemple : `j = "mardi"`). Recopier et compléter l'instruction suivante permettant d'obtenir le numéro du jour de la semaine `n` jours plus tard :

```
numero_jour =(jours.index[ … ] + … )% …
```

3.

a. A partir du dictionnaire `mois`, comment obtenir le nombre de jours du mois de mars ?

b. Le numéro du mois actuel est stocké dans une variable `numero_mois`, écrire le code permettant d'obtenir le nom du mois qu'il sera `x` mois plus tard à partir du dictionnaire `mois`.

Par exemple :

si `numero_mois = 4 et x = 5`, on doit obtenir `"septembre"`

si `numero_mois = 10 et x = 3`, on doit obtenir `"janvier"`

4. On définit une date comme un tuple :

`(nom_jour,numero_jour, numero_mois,annee)` .

a. Sachant que `date = ("samedi", 21, 10, 1995)`, que renvoie `mois[date[2]][1]` ?

b. Ecrire une fonction `jour_suivant(date)` qui prend en paramètre une date sous forme de tuple et qui renvoie un tuple désignant la date du lendemain.

Par exemple :

`jour_suivant( ("samedi", 21, 10, 1995) )` renvoie `("dimanche", 22, 10, 1995)`

`jour_suivant( ("mardi", 31, 10, 1995) )` renvoie `("mercredi", 1, 11, 1995)`

On ne tient pas compte des années bissextiles et on considère que le mois de février comporte toujours 28 jours.

EXERCICE 6 : Cet exercice porte sur les structures de données (dictionnaires).

La cryptographie est un ensemble de techniques permettant de chiffrer un message.

Une technique de cryptographie consiste à mélanger les lettres d'un alphabet et à réécrire le message avec ces permutations. En Python, on peut créer un dictionnaire dans lequel les clés sont les lettres de l'alphabet et les valeurs sont celles de l'alphabet mélangé.

Par exemple, si l'alphabet contient les 4 lettres A, B, C et D, et si le dictionnaire de l'alphabet mélangé est `alpha ={"A": "B", "B": "D", "C": "A", "D": "C"}`, la chaîne de caractères `"BAC"` sera chiffrée `"DBA"`.

Un tel dictionnaire sera appelé **dictionnaire de chiffrement**.

1. On souhaite chiffrer un message écrit avec l'alphabet `A, B, C, D, E, F, G` à l'aide du dictionnaire `alpha ={"A":"B", "B":"D", "C":"A", "D":"C", "E":"F", "F":"G", "G":"E"}`

   a. Quelle est la valeur associée à la clé `"D"` ? En Python, comment l'obtenir ?
   b. Chiffrer la chaîne de caractères `"BAGAGE"` avec le dictionnaire `alpha`.

2. On considère qu'un mot est une chaîne de caractères (un objet de type `str`) écrite uniquement avec les 26 lettres de l'alphabet en majuscule. Par exemple, `"ARBRE"` est un mot et `"L'ARBRE !"` n'est pas un mot à cause des caractères : `"'", " "`(espace) et `"!"`.
Écrire une fonction `chiffrer(mot, alpha)` qui prend en paramètres `mot` un mot et `alpha` un dictionnaire de chiffrement, et qui renvoie une chaîne de caractères chiffrée avec le dictionnaire de chiffrement `alpha`.

3. On souhaite déchiffrer un mot chiffré avec cette méthode.

   a. Si un mot est chiffré avec le dictionnaire de chiffrement `alpha ={"A":"B", "B":"D", "C":"A", "D":"C", "E":"F", "F":"G", "G":"E"}`, donner un dictionnaire permettant de le déchiffrer.

   b. Ecrire une fonction en Python appelée `dico_dechiffrement(dico)` qui prend en paramètre `dico` un dictionnaire de chiffrement et qui renvoie un dictionnaire permettant le déchiffrement. On pourra s'inspirer du code incomplet ci-dessous ou proposer une autre solution :

```
def dico_dechiffrement(dico):
    nouveau = {}
    for lettre in dico :
        code = dico[……]
        nouveau[……] = ……
    return nouveau
```

c. Ecrire une fonction `dechiffre(mot, dico)` qui reçoit un mot chiffré et un dictionnaire de chiffrement et renvoie le mot décodé. On utilisera les fonctions écrites dans les questions précédentes.

4. On souhaite à présent créer un dictionnaire de chiffrement. Ecrire une fonction `dico_chiffrement(alphabet)` qui prend en paramètre `alphabet` un tableau de lettres et qui renvoie un dictionnaire de chiffrement dont les clés sont les lettres du tableau `alphabet` et les valeurs sont les lettres du tableau `alphabet` mélangées.

On pourra utiliser la fonction `shuffle` du module `random` qui mélange en place un tableau. Par exemple, on a :

```
>>> tab = ["A", "B", "C", "D"]
>>> shuffle(tab)
>>> tab
["B", "A", "D", "C"]
```

Exercice 7 : *Cet exercice porte sur la programmation et les algorithmes de tri.*

Au service des urgences d'un hôpital, le triage consiste à classifier ou à déterminer le degré de priorité des patients. Il implique une réévaluation périodique et systématique de ce degré pour les patients en attente.

Dans le système informatique, chaque patient obtient un identifiant à son arrivée en salle d'attente ainsi qu'une priorité dépendant de la gravité potentielle de ses symptômes.
Le patient ayant la priorité 1 est le premier qui doit être pris en charge et deux patients ne peuvent pas avoir la même priorité.

Pour modéliser la salle d'attente en Python, chaque patient est représenté par un tuple composé de son identifiant d'arrivée et de sa priorité. Ainsi, la variable `attente` implémentée ci-dessous représente une salle d'attente de trois patients où, à cet instant, le patient identifié 47 sera le premier à être pris en charge, puis le patient 45 et finalement le patient 49.

```
attente = [(45,2),(47,1),(49,3)]
```

1. Écrire l'instruction qui permet d'insérer dans la liste `attente`, définie ci-dessus, un nouveau patient identifié 50 avec une priorité de 4.

2. Pour optimiser le traitement informatisé de prise en charge des patients, on veut que la salle d'attente soit ordonnée par priorité. On utilise alors la fonction `tri(attente)` donnée ci-dessous qui renvoie la salle d'attente triée dans l'ordre croissant des priorités.

```
def tri(attente) :
    for i in range(len(attente)) :
        pos = i
        mini = attente[i][1]
        for j in range(i, len(attente)) :
            if attente[j][1] < mini :
                pos = j
                mini = attente[j][1]
        temp = attente[i]
        attente[i]= attente[pos]
        attente[pos] = temp
```

a. Quel algorithme de tri est ici implémenté ?

- ✓ le tri fusion
- ✓ le tri par insertion
- ✓ le tri par sélection
- ✓ le tri rapide

b. Quelle est la complexité en temps des tris par insertion et par sélection ?

- ✓ constante : $O(1)$
- ✓ logarithmique : $O(\log n)$
- ✓ linéaire : $O(n)$
- ✓ quadratique : $O(n^2)$

3. On considère dans cette question que la salle d'attente est triée par ordre croissant des priorités.

a. Quand un patient est pris en charge, il faut l'enlever de la salle d'attente. La fonction `quitte(attente)` supprime le patient de priorité 1 de la liste `attente` passée en paramètre. Cette fonction renvoie la nouvelle salle d'attente. On aurait par exemple :

```
>>> quitte([(47, 1), (45, 2), (49, 3)])
[(45, 2), (49, 3)]
```

Recopier et compléter la ligne 14 en définissant une liste en compréhension.

```
13   def quitte(attente) :
14       return [. . . . . .]
```

b. La fonction `quitte(attente)` ne met pas à jour les priorités des patients. Écrire une fonction `maj(attente)` qui met à jour, dans une nouvelle liste, les priorités des patients suite à la prise en charge d'un patient et renvoie cette nouvelle salle d'attente. On aurait par exemple :

```
>>> maj([45, 2), (49, 3)]
[(45, 1), (49, 2)]
```

4.

a. Écrire la fonction `priorite(attente, p)` qui renvoie l'ordre de priorité d'un patient `p` de la liste `attente`. On aurait par exemple :

```
>>> priorite([(45,2),(47,1),(49,3)], 49)
3
```

b. La fonction `revise(attente,p)` renvoie une nouvelle salle d'attente non triée où le patient `p` dont la priorité médicale a évolué reçoit la priorité 1 et les priorités des autres patients sont mises à jour. On aurait par exemple :

```
>>> revise([(45,2),(47,1),(49,3)], 49)
[(45,3),(47,2),(49,1)]
```

Recopier et compléter les lignes 28, 29 du code ci-dessous.

```
23 def revise(attente, p) :
24     nouvelle = []
25     n = priorite(attente, p)
26     for (patient, prio) in attente :
27        if patient == p :
28             nouvelle.append( ……  )
29        elif ……   :
30             nouvelle.append((patient, prio + 1))
31        else :
32             nouvelle.append((patient, prio))
33     return nouvelle
```

Exercice 8 : *Cet exercice traite de programmation, d'algorithmique et de complexité.*

On souhaite rechercher dans un tableau les k plus proches voisins d'un objet donné.

On dispose pour cela d'un tableau `t` non vide contenant des objets d'un même type et d'une fonction `distance` qui renvoie la distance entre deux objets quelconques de ce type.
Étant donné un objet `cible` du même type que ceux du tableau `t`, on cherche à déterminer les indices des k éléments du tableau `t` qui sont les plus proches de cet objet (c'est-à-dire ceux dont la distance à l'objet `cible` est la plus petite).

1. On suppose dans cette question que $k = 1$.
   La fonction `plus_proche_voisin(t, cible)` ci-dessous prend en argument le tableau `t` et l'objet `cible`. Écrire sur votre copie uniquement le bloc d'instructions manquant pour que la fonction renvoie l'indice d'un plus proche voisin de `cible`.

```
def plus_proche_voisin(t, cible) :
    dmin = distance(t[0], cible)
    idx_ppv = 0
    n = len(t)
    for idx in range(1, n) :
                                }  bloc à écrire
    return idx_ppv
```

2. On considère que le coût en temps du bloc manquant est constant. Quelle est la complexité de la fonction `plus_proche_voisin` quand $k = 1$?

Dans la suite, on suppose que $k \geq 1$.

3. Une approche naïve consiste à parcourir le tableau `t` pour trouver l'indice de l'élément le plus proche de `cible`, puis à recommencer pour trouver l'indice du deuxième élément le plus proche de `cible`, et ainsi de suite. Cela implique de parcourir k fois tout le tableau.
   Afin de réduire le nombre d'appels à la fonction `distance`, la stratégie suivante permet de ne parcourir le tableau `t` qu'une seule fois. Lors de ce parcours, on stocke dans une liste `kppv`, initialement vide, les tuples `(idx, d)` où `idx` est l'indice d'un k plus proche voisin de `cible` déjà rencontré et `d` la distance correspondante, triés dans l'ordre décroissant de leur distance à `cible`.
   La fonction `recherche_kppv(t, k, cible)` ci-après renvoie ainsi la liste des tuples `(idx, d)` où `idx` est l'indice d'un k plus proche voisin de `cible` dans le tableau `t` et `d` la distance correspondante.

On admet que la fonction `insertion(kppv, idx, d)` insère le tuple `(idx, d)` dans la liste `kppv` de sorte que celle-ci demeure triée dans l'ordre décroissant des distances.

```
def recherche_kppv(t, k, cible):
    kppv = []
    n = len(t)
    for idx in range(n):
        obj = t[idx]
        if len(kppv)<k:
            insertion(kppv, idx, distance(obj, cible))
        else:
            i0, d0 = kppv[0]
            if distance(obj, cible)<d0:
                kppv.pop(0) # supprime le 1er élément de kppv
                insertion(kppv, idx, distance(obj, cible))
    return kppv
```

a. On remarque qu'il y a plusieurs appels identiques à la fonction `distance(obj,cible)`. Comment ne faire qu'un seul appel de cette fonction ?

b. Expliquer l'intérêt de maintenir la liste `kppv` triée.

c. Écrire une fonction `insertion(kppv, idx, d)` qui insère le tuple `(idx, d)` dans la liste `kppv` préalablement triée en préservant l'ordre décroissant selon l'élément `d`.

On pourra éventuellement utiliser la méthode `insert` dont la documentation, fournie par la commande `help(list.insert)`, est la suivante :

```
insert(self, index, object, /)
 Insère l'objet object avant la position index dans l'objet
appelant référencé par self.
```

Exemple d'utilisation de la méthode `insert` :

```
>>> liste = [4, 2, 8, 9]
>>> liste.insert(1, 3)
>>> liste
[4, 3, 2, 8, 9]
```

EXERCICE 9:

Cet exercice porte sur la programmation de base en Python et sur la structure de données abstraite dictionnaire et ses algorithmes associés.

Lorsque l'énoncé demande la manipulation de la structure de données abstraite liste, on utilisera les `list` en `Python` avec la méthode `append`.

On rappelle que si `mot` est une chaîne de caractère, `mot[i]` renvoie le `(i+1)`^{e} caractère de `mot`.

Exemple : si `mot = 'bonjour'`, `mot[2]` est le caractère `'n'`.

Partie A

Deux mots sont des anagrammes s'ils possèdent exactement les mêmes lettres, mais dans des ordres différents.
Par exemple,

- **AUBE** et **BEAU** sont des anagrammes ;
- **PIRATE**, **PARTIE**, **PATRIE**, **PARITE** sont aussi des anagrammes.

On appelle signature d'un mot la suite de ses lettres données dans l'ordre alphabétique. Si un mot possède plusieurs fois la même lettre, elle est répétée autant de fois qu'elle apparait dans le mot.

Par exemple, la signature du mot **AUBE** est **ABEU**.

1. Donner sans justifier la signature des mots **BEAU**, **PIRATE** et **PATRIE**.

On donne ci-dessous le code `Python` d'une fonction `signature` qui prend en paramètre une chaîne de caractères `mot` et qui renvoie la chaîne de caractères correspondant à la signature de `mot`.

On suppose que l'on dispose d'une fonction `inserer_lettre` qui prend en paramètres une chaîne de caractères `mot`, une chaîne de caractères `lettre` composée d'un seul caractère et un entier `pos`. Cette fonction renvoie une nouvelle chaîne de caractères constituée des lettres de `mot` dans lesquelles la lettre `lettre` aura été insérée à la position `pos`.

Par exemple, l'instruction `inserer_lettre('MARE', 'G', 3)` renvoie `'MARGE'` et l'instruction `inserer_lettre('MER', 'A', 0)` renvoie `'AMER'`.

```
1 def signature(mot):
2     chaine = mot[0]
3     for i in range(1, len(mot)):
4         k = 0
5         while (k < len(chaine)) and (mot[i] > chaine[k]):
6             k = k + 1
7         chaine = inserer_lettre(chaine, mot[i], k)
8         print(chaine)
9     return chaine
```

2. Écrire les affichages successifs produits par la ligne 8, lors de l'appel `signature('PLANTE')`.

3. Écrire une fonction `sont_anagrammes` qui prend en paramètres deux chaînes de caractères `mot1` et `mot2` et qui renvoie `True` si `mot1` et `mot2` sont des anagrammes, ou `False` sinon.

On donne ci-dessous le code Python d'une fonction `mystere`.

```
1 def mystere(liste):
2     dico = {}
3     for mot in liste:
4         s = signature(mot)
5         if s in dico:
6             dico[s].append(mot)
7         else:
8             dico[s] = [mot]
9     return dico
```

4. Écrire le résultat renvoyé par l'instruction suivante :
`mystere(['PIRATE', 'AUBE', 'PATRIE', 'ELEPHANT', 'PARTIE', 'BEAU'])`.

5. Écrire un script Python permettant d'implémenter la fonction `inserer_lettre`.

Partie B

On considère un jeu de lettres constitué de jetons sur lesquels sont inscrits une lettre. Ce jeu consiste à former des mots à partir des lettres que l'on possède puis à placer ces mots sur une grille de façon à ce qu'ils s'intègrent aux mots déjà présents sur la grille.

Chaque joueur dispose de 7 jetons (donc de 7 lettres), tirés au hasard. Par exemple, avec les jetons portant les lettres A, A, E, R, T, U, U, les mots RATE, TARE, RATEAU, TAUREAU sont des mots possibles.

Un joueur n'a donc pas l'obligation de former un mot de 7 lettres, le mot peut être plus court. En revanche, le mot devra figurer dans une liste de mots fournie par le jeu.

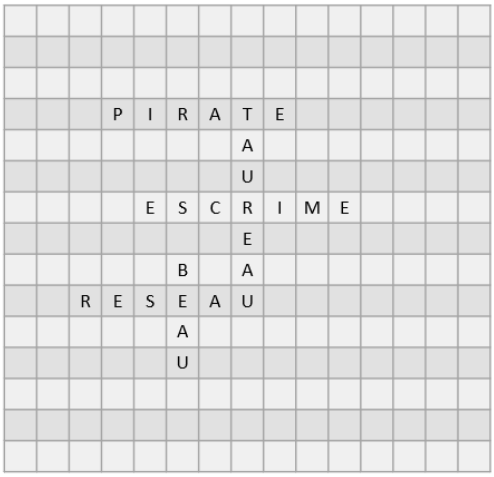


Figure 1. Exemple de partie

Chaque lettre est pondérée par un score qui dépend de la difficulté à utiliser cette lettre dans un mot. Pour cela, on utilisera un dictionnaire `score_lettres` indiquant le nombre de points correspondant à chaque lettre. Ce dictionnaire est défini dans le corps de la fonction `score_mot` ci-dessous.

La fonction `score_mot` prend en paramètre une chaîne de caractères `mot` et renvoie un entier correspondant au score de `mot` d'après les informations du dictionnaire `score_lettres`.
Par exemple, l'instruction `score_mot('PIRATE')` renvoie 8.

```
1 def score_mot(mot):
2     score_lettres = {'A' : 1, 'B' : 3, 'C' : 3, 'D' : 2, 'E' : 1,
                       'F' : 4, 'G' : 2, 'H' : 4, 'I' : 1, 'J' : 8,
                       'K' : 10, 'L' : 1, 'M' : 2, 'N' : 1, 'O' : 1,
                       'P' : 3, 'Q' : 8, 'R' : 1, 'S' : 1, 'T' : 1,
                       'U' : 1, 'V' : 4, 'W' : 10, 'X' : 10,
                       'Y' : 10, 'Z' : 10}
3     ...
```

6. Écrire le script de la fonction `score_mot` à partir de la ligne 3.

7. Écrire une fonction `meilleur_mot` qui prend en paramètre une liste de chaînes de caractères `liste_mots` et qui renvoie la chaîne de caractères de `liste_mots` dont le score est le plus élevé.
   Par exemple, avec la liste de mots
   `L = ['PIRATE', 'AUBE', 'SOIGNER', 'CHAPEAU', 'DRAPEAU']`,
   l'instruction `meilleur_mot(L)` renvoie `'CHAPEAU'`.

Dans toute la suite de l'exercice, on appellera chevalet le support sur lequel le joueur dispose ses jetons.

En Python, le contenu d'un chevalet sera représenté par une liste de caractères. Par exemple, si le joueur dispose les lettres A, Z, T, R, N, E et P, le chevalet sera représenté par la liste : `['A', 'Z', 'T', 'R', 'N', 'E', 'P']`.

À chaque fois que le joueur dépose des lettres sur la grille, il est nécessaire de mettre à jour son chevalet.

Par exemple, si le chevalet du joueur correspond à la liste `['A', 'E', 'S', 'E', 'E', 'S', 'R']` et que le joueur dépose les lettres R, E, S, E, A, alors son chevalet devient `['E', 'S']`.

Cette mise à jour du chevalet est assurée par la fonction `chevalet_apres_depot` qu'on ne demande pas de programmer.

Par exemple, l'appel `chevalet_apres_depot(['A', 'E', 'S', 'E', 'E', 'S', 'R'], ['R', 'E', 'S', 'E', 'A'])` renvoie la liste `['E', 'S']`.

En début de partie, puis à chaque fois que le joueur a déposé des lettres sur la grille, il faut compléter son chevalet pour qu'il contienne 7 jetons.

Pour cela, on donne ci-après le code `Python` incomplet d'une fonction `pioche` qui prend en paramètre une liste de caractères `liste_lettres` éventuellement vide. Cette fonction permet de compléter le chevalet d'un joueur en y ajoutant autant de lettres que nécessaire.

Par exemple,

- en début de partie, l'instruction `pioche([])` permet de renvoyer une liste de 7 caractères aléatoires ;
- en cours de partie, si, après avoir déposé des lettres sur la grille, le joueur dispose encore des lettres C, A, E, T et C, alors l'appel `pioche(['C', 'A', 'E', 'T', 'C'])` peut renvoyer la liste `['C', 'A', 'E', 'T', 'C', 'S', 'U ']` : cela signifie que le joueur a tiré, au hasard, un jeton portant la lettre S et un jeton portant la lettre U.

```
1  from random import randint

2  def pioche(liste_lettres):
3      lettres = ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H', 'I',
                  'J', 'K', 'L', 'M', 'N', 'O', 'P', 'Q', 'R',
                  'S', 'T', 'U', 'V', 'W', 'X', 'Y', 'Z']
4      m = len(liste_lettres)
5      while ...:
6          # Tirage aléatoire d'une lettre
7          nb_alea = randint(0, 25)
8          tirage = lettres[...]
9          liste_lettres.append(tirage)
10         m = m + 1
11     return liste_lettres
```

8. Recopier et compléter les lignes 5 et 8 de la fonction `pioche`.

Le fichier `mots_valides.txt` contient un peu plus de 142 000 mots, écrits en minuscules et sans accent à raison d'un mot par ligne. Chaque mot est donc séparé

par un caractère ‘retour à la ligne’ représenté par `\n` en Python. On donne ci-dessous les 5 premières lignes de ce fichier.

```
abaissement
abaisser
abandon
abandonnant
abandonne
```

La méthode `strip` appliquée à une chaîne de caractères `mot` permet, entre autres, de renvoyer la même chaine de caractères privé du caractère `\n` si celui-ci est en fin de la chaîne de caractères `mot`. On donne une illustration du fonctionnement de cette méthode ci-dessous :

```
>>> mot = 'abaissement\n'
>>> mot = mot.strip()
>>> mot
'abaissement'
```

Dans le jeu, un mot constitué avec les lettres du chevalet est considéré comme valide uniquement si ce mot est un des mots de ce fichier texte.

On donne le code Python suivant qui permet de créer le dictionnaire des anagrammes des mots contenus dans le fichier `mots_valides.txt`. On suppose que ce fichier est enregistré dans le même répertoire que le script python.

```
1  def lecture(mon_fichier):
2      fichier = open(mon_fichier, 'r', encoding = 'UTF-8')
3      liste_mots = []
4      for ligne in fichier:
5          mot = ligne.strip()
6          liste_mots.append(mot)
7      fichier.close()
8      return liste_mots

9  L = lecture('mots_valides.txt')
10 dico = mystere(L)
```

9. Expliquer ce que signifie la valeur `'r'` prise par l’un des arguments de la fonction `open`, ligne 2.

Après exécution de ce script, l’instruction `dico['EIMPRS']` renvoie la liste des anagrammes `['IMPERS', 'MEPRIS', 'PERMIS', 'PRIMES', 'PRISME']`.

On dispose également d’une fonction `signatures_depuis_tirage` qui, à partir d’un tirage de lettres donné sous la forme d’une liste de caractères, renvoie la liste des signatures de tous les mots construits à partir de ce tirage.

Par exemple, l’instruction suivante, avec une liste de 4 lettres en argument, `signatures_depuis_tirage(['A', 'B', 'C', 'D'])` renvoie la liste

`['A', 'B', 'AB', 'C', 'AC', 'BC', 'ABC', 'D', 'AD', 'BD', 'ABD', 'CD', 'ACD', 'BCD', 'ABCD']`.

On donne ci-dessous le début d'un script Python.

```
1 chevalet = pioche([])
2 mots_possibles = []
3 liste_signatures = signatures_depuis_tirage(chevalet)
4 ...
```

10. Recopier et compléter ce script afin de répondre aux trois points suivants :

- déterminer tous les mots du fichier `mots_valides.txt` qu'il est possible d'écrire à partir de la liste de lettres contenues dans la variable `chevalet` ;
- afficher le mot dont le score est le plus important ;
- le programme devra compléter le chevalet pour qu'il contienne à nouveau 7 jetons, en supposant que le joueur a déposé sur la grille les lettres permettant de constituer le mot dont le score est le plus élevé.

Exercice 10 : ***Cet exercice porte sur les tableaux et sur la programmation de base en Python.***

On rappelle que `len` est une fonction qui prend un tableau en paramètre et renvoie sa longueur. C'est-à-dire le nombre d'éléments présents dans le tableau.
Exemple : `len([12, 54, 34, 57])` vaut 4.

Le but de cet exercice est de programmer différentes réductions pour un site de vente de vêtements en ligne.
On rappelle que si le prix d'un article avant réduction est de x euros,

- son prix vaut $0,5x$ si on lui applique une réduction de 50%,
- son prix vaut $0,6x$ si on lui applique une réduction de 40%,
- son prix vaut $0,7x$ si on lui applique une réduction de 30%,
- son prix vaut $0,8x$ si on lui applique une réduction de 20%,
- son prix vaut $0,9x$ si on lui applique une réduction de 10%.

Dans le système informatique du site de vente, l'ensemble des articles qu'un client veut acheter, appelé *panier*, est modélisé par un tableau de flottants.
Par exemple, si un client veut acheter un pantalon à 30,50 euros, un tee-shirt à 15 euros, une paire de chaussettes à 6 euros, une jupe à 20 euros, une paire de collants à 5 euros, une robe à 35 euros et un short à 10,50 euros, le système informatique aura le tableau suivant :

`tab = [30.5, 15.0, 6.0, 20.0, 5.0, 35.0, 10.5]`.

1. (a) Écrire une fonction Python `total_hors_reduction` ayant pour argument le tableau des prix des articles du panier d'un client et renvoyant le total des prix de ces articles.

   (b) Le site de vente propose la promotion suivante comme offre de bienvenue : 20% de réduction sur le premier article de la liste, 30% de réduction sur le deuxième article de la liste (s'il y a au moins deux articles) et aucune réduction sur le reste des articles (s'il y en a).
   Recopier sur la copie et compléter la fonction Python `offre_bienvenue` prenant en paramètre le tableau `tab` des prix des articles du panier d'un client et renvoyant le total à payer lorsqu'on leur applique l'offre de bienvenue.

```
def offre_bienvenue(tab):
    """ tableau -> float """
    somme=0
    longueur=len(tab)
    if longueur > 0 :
        somme=tab[0]*...
    if longueur > 1 :
        somme=somme + ...
    if longueur > 2 :
        for i in range(2,longueur):
            somme=...
    return ...
```

Pour toute la suite de l'exercice, on pourra utiliser la fonction `total_hors_reduction` même si la question 1 n'a pas été traitée.

2. Lors de la période des soldes, le site de vente propose les réductions suivantes :
   - si le panier contient 5 articles ou plus, une réduction globale de 50%,
   - si le panier contient 4 articles, une réduction globale de 40%,
   - si le panier contient 3 articles, une réduction globale de 30%,
   - si le panier contient 2 articles, une réduction globale de 20%,
   - si le panier contient 1 article, une réduction globale de 10%.

   Proposer une fonction Python `prix_solde` ayant pour argument le tableau `tab` des prix des articles du panier d'un client et renvoyant le total des prix de ces articles lorsqu'on leur applique la réduction des soldes.

3. (a) Écrire une fonction `minimum` qui prend en paramètre un tableau `tab` de nombres et renvoie la valeur minimum présente dans le tableau.

   (b) Pour ses bons clients, le site de vente propose une offre promotionnelle, à partir de 2 articles achetés, l'article le moins cher des articles commandés est offert.

   Écrire une fonction Python `offre_bon_client` ayant pour paramètre le tableau des prix des articles du panier d'un client et renvoyant le total à payer lorsqu'on leur applique l'offre bon client.

4. Afin de diminuer le stock de ses articles dans ses entrepôts, l'entreprise imagine faire l'offre suivante à ses clients : en suivant l'ordre des articles dans le panier du client, elle considère les 3 premiers articles et offre le moins cher, puis les 3 suivants et offre le moins cher et ainsi de suite jusqu'à ce qu'il reste au plus 2 articles qui n'ont alors droit à aucune réduction.

   Exemple : Si le panier du client contient un pantalon à 30,50 euros, un tee-shirt à 15 euros, une paire de chaussettes à 6 euros, une jupe à 20 euros, une paire de collants à 5 euros, une robe à 35 euros et un short à 10,50 euros, ce panier est représenté par le tableau suivant :

   ```
   tab = [30.5, 15.0, 6.0, 20.0, 5.0, 35.0, 10.5]
   ```

   Pour le premier groupe (le pantalon à 30,50 euros, le tee-shirt à 15 euros, la paire de chaussettes à 6 euros), l'article le moins cher, la paire de chaussettes à 6 euros, est offert. Pour le second groupe (la jupe à 20 euros, la paire de collants à 5 euros, la robe à 35 euros), la paire de collants à 5 euros est offerte.

   Donc le total après promotion de déstockage est 111 euros.

   On constate que le prix après promotion de déstockage dépend de l'ordre dans lequel se présentent les articles dans le panier.

   (a) Proposer un panier contenant les mêmes articles que ceux de l'exemple mais ayant un prix après promotion de déstockage différent de 111 euros.

   (b) Proposer un panier contenant les mêmes articles mais ayant le prix après promotion de déstockage le plus bas possible.

   (c) Une fois ses articles choisis, quel algorithme le client peut-il utiliser pour modifier son panier afin de s'assurer qu'il obtiendra le prix après promotion de déstockage le plus bas possible ? On ne demande pas d'écrire cet algorithme.

Exercice *11 :*

Cet exercice porte sur la notion de chaînes de caractères, de tableau et sur la programmation de base en Python.

On appelle palindrome un texte dont l'ordre des lettres reste le même, qu'on le lise de la droite vers la gauche ou de la gauche vers la droite.
Par exemple, un mot à une lettre est un palindrome, `"BOB"` est un palindrome, tout comme `"LAVAL"`.
Le mot `""` est considéré comme un palindrome.
On souhaite programmer une fonction `palindrome` qui prend en paramètre une chaîne de caractères `txt`. Cette fonction renvoie `True` si la chaîne de caractères `txt` est un palindrome, `False` sinon.

On rappelle que :

- La fonction `len` prend une chaîne de caractères en paramètre et renvoie sa longueur, c'est-à-dire le nombre de lettres constituant la chaîne de caractères.
  Exemple : `len("bonjour")` vaut 7.
- Si `txt` est une chaîne de caractères, `txt[i]` est la lettre de `txt` d'indice i. Attention, la première lettre a pour indice 0.
  Exemple : si `txt = "bonjour"` alors `txt[2]` désigne `"n"`.

1. On donne ci-dessous une implémentation récursive incomplète pour la fonction `palindrome`.
   L'opération `+` à la ligne 8 permet de concaténer deux chaînes de caractères.
   Exemple : Si `txt1 = "bon"` et `txt2 = "jour"`, l'instruction `txt1 + txt2` renvoie la chaîne de caractères `"bonjour"`.

```
1 def palindrome(txt):
2     """ Str -> Bool """
3     ----------------:
4     ----------------
5     taille = len(txt)
6     interieur = ""
7     for i in range(1, taille - 1):
8         interieur = interieur + txt[i]
9     return (txt[0] == txt[taille - 1]) and (palindrome(interieur))
```

   (a) Choisir parmi les propositions ci-dessous celle qui qui convient pour compléter la fonction palindrome (ligne 3 et 4) :

Proposition 1 :

```
1 if len(txt) <= 2:
2     return True
```

Proposition 3 :

```
1 if len(txt) < 2:
2     return True
```

Proposition 2 :

```
1 if len(txt) < 2:
2     return False
```

Proposition 4 :

```
1 if len(txt) <= 2:
2     return False
```

(b) Lors de l'appel de `palindrome("bonjour")`, indiquer les valeurs, à la ligne 9, de : `txt[0]`, `txt[taille - 1]` et `interieur`.

2. Proposer deux tests pour cette fonction qui permettent de tester deux cas de figure différents en justifiant ce choix.
3. Écrire une version non récursive de la fonction `palindrome`.
4. On étudie dans cet exercice des chaînes de caractères utilisant uniquement les lettres `"A"`, `"T"`, `"C"` et `"G"`.

   Exemple : `"AA"`, `"CAT"` et `"CCGATACG"`.

   On associe à chacune de ces lettres une autre lettre appelée lettre complémentaire selon le tableau suivant :

Lettre	`"A"`	`"T"`	`"G"`	`"C"`
Lettre complémentaire	`"T"`	`"A"`	`"C"`	`"G"`

   On obtient le complémentaire d'un mot en remplaçant chacune de ses lettres par sa lettre complémentaire.

   Exemple : Le complémentaire à `"GAATTC"` est `"CTTAAG"`.

   (a) Écrire une fonction en Python, nommée `complementaire`, qui prend en paramètre une chaîne de caractères `txt` écrite uniquement avec les lettres `"A"`, `"C"`, `"G"` et `"T"`. Cette fonction renvoie la chaîne de caractères complémentaire de `txt`.

   Exemple : l'appel `complementaire("GAATTC")` renvoie `"CTTAAG"`.

   (b) Une chaîne de caractères `txt` est dite palindromique si la concaténation de `txt` avec son complémentaire est un palindrome.

   Exemples :

   `"GAATTC"` est palindromique car `"GAATTC" + "CTTAAG" = "GAATTCCTTAAG"` est un palindrome.

   `"GAAT"` n'est pas palindromique car `"GAAT" + "CTTA" = "GAATCTTA"` n'est pas un palindrome.

   Déterminer si la chaîne de caractères `"GATCGT"` est palindromique.

   (c) Écrire une fonction `est_palindromique` prenant comme paramètre une chaîne de caractères `txt`. Cette fonction doit renvoyer `True` s'il s'agit d'une séquence palindromique, `False` sinon.

Exercice *12 :*

Cet exercice porte sur les tableaux à deux dimensions et la programmation python en général.

Partie A

Dans cette partie, on s'intéresse à un dessin pixelisé en noir et blanc dans lequel chaque ligne est obtenue séquentiellement à partir de la ligne juste au-dessus d'elle.
Au départ, on dispose donc de la ligne du haut et on déduit les lignes en-dessous les unes après les autres.
Les règles sont les suivantes :

— pour chaque ligne, le pixel complètement à droite et le pixel complètement à gauche sont blancs.

— pour les autres pixels, un pixel est noir si les deux pixels à droite et gauche du pixel juste au-dessus de ce pixel ne sont pas de la même couleur, sinon il est blanc.

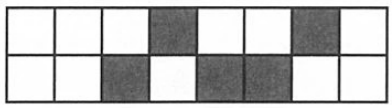
a		b
	x	

Le pixel noté "x" est donc blanc si les pixels notés "a" et "b" sont de la même couleur ou noir sinon.

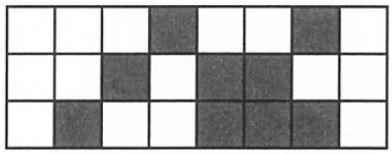
1. (a) Dans cette question uniquement, on considère un dessin de cinq lignes et huit colonnes.
La première ligne est colorée comme ci-dessous :



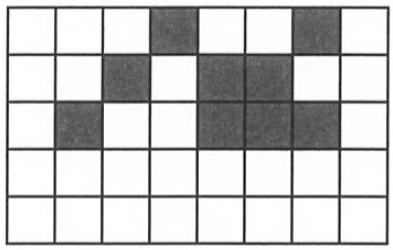
En respectant les règles de coloration, on complète la figure en ajoutant la deuxième ligne :



De même, en ajoutant la troisième ligne :



Recopier et colorier les deux dernières lignes du dessin ci-dessous en respectant les règles.



Dans la suite de cet exercice :

— les lignes sont numérotées du haut vers le bas, en commençant à 0 ;

— les colonnes sont numérotées de la gauche vers la droite, en commençant à 0.

(b) On considère le pixel de la ligne 4 et de la colonne 1.

— Indiquer sur quelle ligne et sur quelle colonne se trouve le pixel situé sur la ligne juste au-dessus, à sa gauche.

— Même question, pour le pixel situé sur la ligne juste au-dessus, à sa droite.

(c) Plus généralement, on considère le pixel de la ligne `li` et de la colonne `co` et situé ni sur la première ligne, ni sur la colonne tout à gauche, ni sur la colonne tout à droite.

— Indiquer sur quelle ligne et sur quelle colonne se trouve le pixel situé sur la ligne juste au-dessus, à sa gauche.

— Même question, pour le pixel situé sur la ligne juste au dessus, à sa droite.

2. On dispose d'un tableau `image` à deux dimensions (m lignes et 8 colonnes) qui contient uniquement des 0 ou des 1. `image[li][co]` vaut 1 si le pixel de la ligne `li` et la colonne `co` est noir et 0 sinon.
Pour chaque valeur de `li`, `image[li][0] = 0` et `image[li][7] = 0`.

(a) On suppose que le tableau `image` a été rempli en respectant les règles de coloration jusqu'à la ligne `li-1` et contient des 0 partout ailleurs. On suppose de plus que $0 <$`co`< 7.
On cherche à connaître quelle valeur la case `image[li][co]` doit prendre si on respecte les règles de coloration. Donner les conditions portant sur les valeurs de la ligne `li-1` qui doivent être satisfaites pour que `image[li][co]` prenne la valeur `1`.

(b) On suppose toujours que le tableau `image` a été rempli en respectant les règles de coloration jusqu'à la ligne `li-1` et contient des 0 partout ailleurs.
Recopier et compléter la fonction Python `remplir_ligne` qui prend en paramètre le tableau `image` et un entier `li`. Cette fonction affecte à chaque case de la ligne `li` de `image`, la valeur correspondant à la couleur du pixel, en respectant les règles de coloration (dans le code de la fonction, les trois points (...) peuvent correspondre à une ou plusieurs lignes de programme).

```
1 def remplir_ligne(image,li) :
2     image[li][0] = 0
3     image[li][7]= 0
4     for ... in range( ... , ... ) :
5         ...
```

(c) La première ligne du tableau `image` est remplie correctement.
Écrire une fonction `remplir` qui prend en paramètre le tableau `image`. Cette fonction modifie les valeurs des cases du tableau `image` des lignes restantes.

Partie B

À un tableau de taille 8 contenant des 0 ou des 1, on associe l'entier dont la représentation binaire est donnée par ce tableau.
Par exemple, le tableau `[0, 0, 0, 1, 0, 0, 1, 0]` représente le nombre binaire `00010010`, correspondant à 18 en base 10.

1. (a) Soit le tableau `[0, 0, 1, 0, 1, 1, 0, 0]`. Déterminer la représentation en base 10 de l'entier correspondant.

   (b) Écrire la fonction Python `conversion2_10` qui prend en paramètre un tableau `tab` de taille 8 composé de 0 et 1, et qui renvoie l'entier correspondant, écrit en base 10.

   Exemple : `conversion2_10([0, 0, 0, 1, 0, 0, 1, 0])` vaut `18`.

   (c) Réciproquement, donner le tableau associé à l'entier 78 dont l'écriture en base 10 est 78.

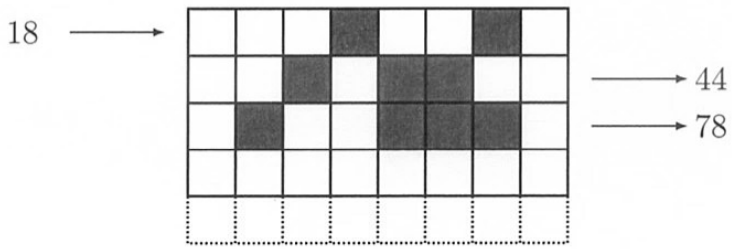
Dans la suite, on suppose qu'on dispose de la fonction Python `conversion10_2`, qui prend en paramètre un entier n compris entre 0 et 255, et qui renvoie le tableau `tab` correspondant.

Exemple : `conversion10_2(18)` vaut `[0, 0, 0, 1, 0, 0, 1, 0]`.

2. À partir d'un entier n, on construit une liste de k entiers en utilisant l'image étudiée dans la partie A, de la façon suivante :

   — l'entier n fournit la première ligne de l'image sous forme d'un tableau de taille 8, représentant son écriture binaire ;

   — chaque ligne suivante de l'image est obtenue selon les règles appliquées dans la partie A et permet de déterminer un entier de la liste.



Par exemple, l'entier 18 fournit la liste 44, 78, ...

   (a) On rappelle que pour chaque ligne de l'image, le pixel complètement à gauche et le pixel complètement à droite restent blancs. En déduire une ou des précondition(s) sur l'entier n.

   (b) Recopier et compléter la fonction `generer` qui prend en paramètres deux entiers `n` et `k`, et renvoie le tableau `tab`, de taille k, contenant les k entiers obtenus à partir de n (dans le code de la fonction, les trois points (...) peuvent correspondre à une ou plusieurs lignes de programme).

```
1 def generer(n, k):
2     tab = [None for i in range(k)]
3     image = [[0 for j in range(8)] for i in range(k+1)]
4     ...
5     return tab
```

Exercice 13 :

Thèmes abordés : algorithmique – chaînes de caractères – complexité.

Cet exercice propose l'étude d'un algorithme de détection de palindrome. On rappelle les définitions suivantes :

Définition 1 : un mot est un **palindrome** s'il peut se lire aussi bien dans les deux sens, par exemple le mot « kayak »

Définition 2 : un **variant de boucle** est une suite de valeurs d'entiers positifs strictement décroissante

On considère la fonction `palindrome1` qui renvoie un booléen et dont le paramètre `mot` est une chaîne de caractères de longueur `n`.

```
1|   Fonction palindrome1(mot) :
2|     Variables : i,j : ENTIER ; p : BOOLEEN
3|     i ← 0
4|     j ← longueur(mot)-1
5|     p ← Vrai
6|     tant que i≤j
7|       Si mot[i] ≠ mot[j]
8|         p ← Faux
9|       FinSi
10|      i ← i+1
11|      j ← j-1
12|    Fin tant que
13|    Renvoie p
```

1. Exécuter ligne après ligne cette fonction avec comme argument la chaîne de caractères "rotor" en recopiant le tableau suivant. L'étape 1 correspond à la première exécution de la boucle. Attention : toutes les colonnes ne sont pas forcément à remplir intégralement.

	Initialisation	Etape 1	Etape 2	Etape 3	Etape 4	…
`i ≤ j`						
`mot[i] ≠ mot[j]`						
`i`	0					
`j`	4					
`p`	Vrai					

2.

a. Combien de comparaisons de caractères sont réalisées pour cet algorithme avec le mot "rotor" ?

b. Combien de comparaisons de caractères sont réalisées pour un mot de longueur `n` dans le pire des cas ?

3. Montrer que la quantité `(j-i)` est un variant de la boucle `tant que`. En déduire que cette boucle se termine.

4. Combien de fois la boucle est-elle exécutée avec le mot "`routeur`" ? Proposer une amélioration de l'algorithme de la fonction `palindrome1` que l'on appellera `palindrome2` permettant d'éviter les tours de boucle inutiles. Justifier votre proposition.

EXERCICE *14 :*

En informatique, chaque pixel d'une image numérique est affiché sur un écran standard par synthèse additive du rouge, du vert et du bleu : c'est le système colorimétrique RVB. Chacune des trois composantes RVB d'un pixel est stockée sur un octet.

Pour représenter une image, un fichier au format *Windows bitmap* (BMP) contient un entête, suivi des valeurs des composantes RVB de chacun des pixels, écrites les unes à la suite des autres.

1. On considère une image de hauteur 1000 pixels et de largeur 1500 pixels.
   Combien de mégaoctets faut-il pour représenter l'ensemble des pixels de cette image en omettant l'entête ?

2. En Python, les images sont généralement représentées à l'aide de tableaux de tableaux.
   - les composantes RVB d'un pixel sont stockées dans un `tuple` de trois entiers compris entre 0 et 255 ;
   - pour chaque ligne de l'image, les composantes RVB des pixels sont stockées dans un tableau ;
   - l'image est alors représentée par un tableau `img` contenant tous les tableaux précédents ; par convention, l'image vide est représentée par le tableau contenant un tableau vide `[[]]`.

   Ainsi, si l'image `img` a une hauteur de 1000 pixels et largeur de 1500 pixels,

```
>>> len(img)
1000
>>> len(img[0])
1500
```

   La position de chaque pixel dans une image `img` est repérée par un couple d'entiers `(i, j)` qui sont respectivement les indices de ligne et de colonne du pixel dans `img`.

   Encadrer `i` et `j` lorsque l'image `img` a pour hauteur `n` et largeur `m`.

Lorsque `i` et `j` vérifient ces encadrements, on dira qu'ils sont **compatibles** avec la taille de l'image.

On s'intéresse dans la suite de cet exercice à l'algorithme de recadrage intelligent (*seam carving*), qui permet de réduire la largeur d'une image tout en conservant ses principales caractéristiques graphiques.

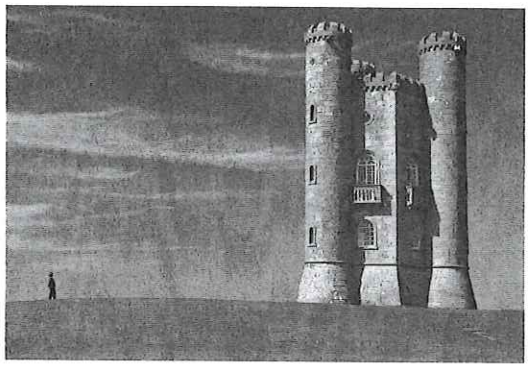
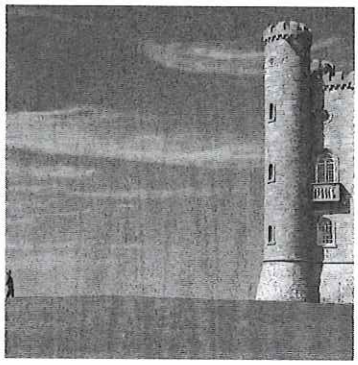
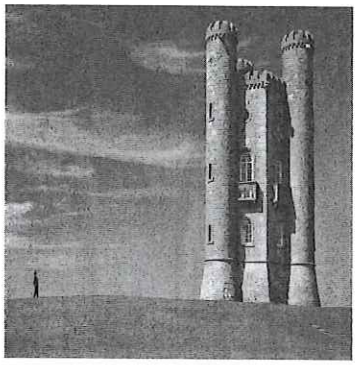


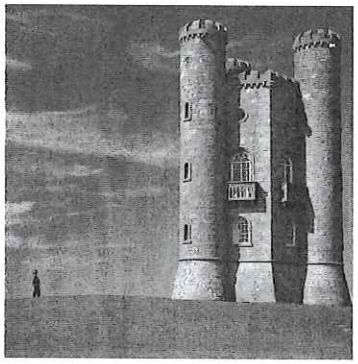
Image dont on souhaite réduire la largeur



Découpe de l'image
une partie du château n'est plus visible



Redimensionnement
le château est déformé

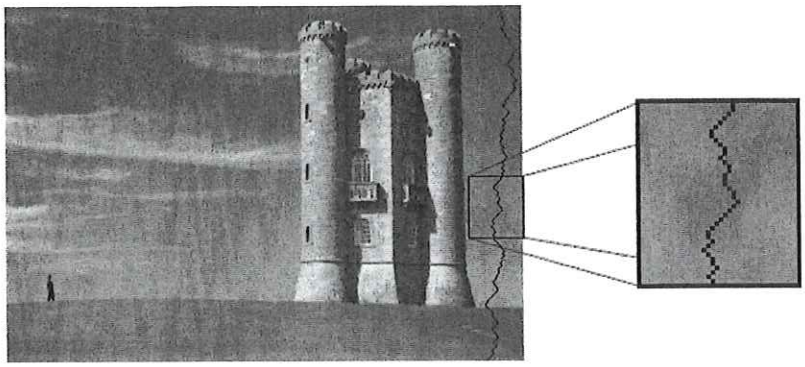


Seam carving
les aspects des principaux éléments graphiques sont conservés

Source des illustrations : https://en.wikipedia.org/wiki/Seam_carving

L'algorithme de recadrage intelligent s'appuie sur la suppression de pixels « bien choisis » dans l'image. Pour déterminer ces pixels, il recherche ce que l'on appellera la « couture de moindre énergie ».

Une couture est une suite de pixels adjacents allant du haut au bas de l'image. On associe à chaque pixel de l'image un nombre positif appelé énergie du pixel. L'énergie d'une couture est alors la somme des énergies des pixels qui la composent.



Une couture et un détail de cette couture.

L'algorithme de recadrage intelligent d'une image de dimensions `n` × `m` (hauteur `n`, largeur `m`) détermine alors la couture de plus basse énergie et supprime de l'image tous les pixels présents dans la couture. L'image résultante est de dimensions `n` × `(m - 1)` (hauteur `n`, largeur `m - 1`). On recommence le procédé jusqu'à obtenir la largeur souhaitée.

3. On suppose que l'on dispose d'une fonction `energie(img, i, j)`, qui prend en paramètres une image `img`, un indice de ligne `i`, un indice de colonne `j` et renvoie l'énergie du pixel à la position `(i, j)` de l'image.
On décide de calculer au préalable les énergies des différents pixels de l'image. La fonction `calcule_tab_energie` prend comme paramètre une image `img` et renvoie le tableau de tableaux des énergies correspondantes. Ainsi `tab_energie[i][j]` contiendra à la fin de l'exécution l'énergie du pixel à la position `(i, j)`.

```
def calcule_tab_energie(img):
    n = len(img)
    m = len(img[0])
    tab_energie = []
    for i in range(n):
        ligne = []
        for j in range(m):
            e = energie(img, i, j)
            ligne.append(e)
        tab_energie.append(ligne)
    return tab_energie
```

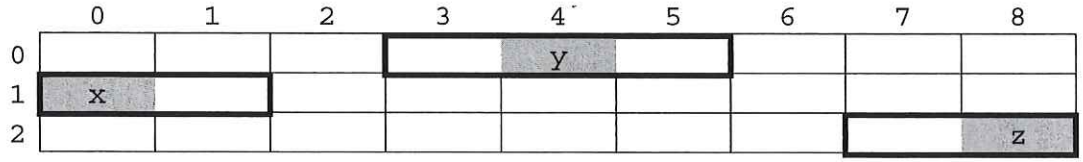
   a. Quel est le type de la variable `tab_energie` renvoyée par la fonction `calcule_tab_energie(img)` ?

   b. Exprimer en fonction de `n` et de `m` le nombre d'appels à la fonction `energie`.

4. On représente en Python une couture à l'aide d'un tableau de couples d'indices `(i, j)` compatibles avec la taille de l'image. Chaque couple `(i, j)` repère la position dans l'image d'un pixel de la couture.
On rappelle que l'énergie d'une couture est la somme des énergies des pixels qui la composent.

Écrire une fonction `calcule_energie(couture, tab_energie)`, qui prend en paramètres un tableau `couture` représentant une couture ainsi que le tableau des énergies `tab_energie` renvoyé par la fonction `calcule_tab_energie`. Cette fonction doit renvoyer l'énergie de la couture passée en argument.

5. Écrire la fonction `indices_proches(m, i, j)`, qui prend en paramètres `m` le nombre de colonnes de l'image, `i` un indice de ligne, `j` un indice de colonne, et renvoie parmi les positions des pixels `(i,j-1)`, `(i,j)` et `(i,j+1)` celles qui sont compatibles avec les dimensions de l'image.
On ne tiendra pas compte de l'ordre des éléments de la liste renvoyée.
Par exemple, on a repéré dans l'image de dimension 3×9 ci-dessous trois pixels `x`, `y`, `z`.




```
>>> indices_proches(9, 1, 0) # pixels voisins de x
[(1,0), (1,1)]
>>> indices_proches(9, 0, 4) # pixels voisins de y
[(0,3), (0,4), (0,5)]
>>> indices_proches(9, 2, 8) # pixels voisins de z
[(2,7), (2,8)]
```

6. On appellera couture d'indice `j` une couture dont la position du premier pixel est `(0, j)`. Afin de déterminer une couture de basse énergie parmi toutes les coutures d'indice `j`, on met en œuvre une stratégie de type glouton.
Pour cela, on construit la couture du haut vers le bas en sélectionnant à chaque étape le pixel d'énergie minimale parmi ceux situés immédiatement en bas à gauche, immédiatement en bas ou immédiatement en bas à droite lorsque cela est possible.

Par exemple, on donne ci-dessous un tableau `tab_energie` d'une image de dimension `4 × 3`. Les pixels de la couture d'indice `j = 1` y ont été grisés.

	0	1	2
0	3	4	2
1	4	5	6
2	3	2	1
3	7	8	6

Le premier pixel de la couture a pour position `(0, 1)`
Le 2e pixel sera choisi parmi `[(1,0), (1,1), (1,2)]`
Le 3e pixel sera choisi parmi `[(2,0), (2,1)]`
Le 4e pixel sera choisi parmi `[(3,0), (3,1), (3,2)]`

La couture d'indice 1 est donc : `[(0,1), (1, 0), (2, 1), (3, 2)]`

La fonction `calcule_couture(j, tab_energie)`, ci-dessous a pour paramètres un entier `j` et le tableau des énergies `tab_energie`. Cette fonction doit renvoyer une liste de tuples représentant la couture d'indice `j` construite selon le procédé décrit précédemment.
On pourra utiliser la fonction `indices_proches`, ainsi que la fonction `min_energie(tab_energie, indices_pixels)`, qui prend en paramètres le tableau d'énergie `tab_energie` et la liste de positions de pixels `indices_pixels`. La fonction `min_energie` renvoie le tuple `(i,j)` de la position du pixel de moindre énergie.

Recopier et compléter le code de la fonction `calcule_couture` ci-dessous :

```
def calcule_couture(j, tab_energie) :
    n = len(tab_energie)      # hauteur de l'image
    m = len(tab_energie[0]) # largeur de l'image
    couture = []
    couture.append((0, j))    # premier pixel de la couture
    for i in range(1, n):     # i est l'indice de la ligne du
                              # prochain pixel dans la couture
        ...                   # plusieurs lignes possibles

    return couture
```

7. Le paramètre `j` de la fonction précédente peut prendre toutes les valeurs entières comprises entre `0` (inclus) et `m = len(tab_energie[0])` (exclu).
Écrire une fonction `meilleure_couture(tab_energie)`, qui renvoie une couture d'énergie minimale parmi celles obtenues à l'aide de la fonction `calcule_couture`.
L'algorithme consistera à parcourir tous les indices `j` et ne garde qu'une couture d'indice `j` qui soit d'énergie minimale. Pour cela, on pourra utiliser les fonctions `calcule_couture` et `calcule_energie` définies auparavant.

EXERCICE 15 :

Cet exercice porte sur les listes, les dictionnaires et la programmation de base en Python.

Pour son évaluation de fin d'année, l'institut d'Enseignement Néo-moderne (EN) a décidé d'adopter le principe du QCM. Chaque évaluation prend la forme d'une liste de questions numérotées de 0 à 19. Pour chaque question, 5 réponses sont proposées. Les réponses sont numérotées de 1 à 5. Exactement une réponse est correcte par question et chaque candidat coche exactement une réponse par question. Pour chaque évaluation, on dispose de la correction sous forme d'une liste `corr` contenant pour chaque question, la bonne réponse ; c'est-à-dire telle que `corr[i]` est la bonne réponse à la question `i`. Par exemple, on présente ci-dessous la correction de l'épreuve 0 :

```
corr0 = [4, 2, 1, 4, 3, 5, 3, 3, 2, 1, 1, 3, 3, 5, 4, 4, 5, 1, 3, 3].
```

Cette liste indique que pour l'épreuve 0, la bonne réponse à la question 0 est 4, et que la bonne réponse à la question 19 est 3.

Avant de mettre une note, on souhaite corriger les copies question par question ; c'est-à-dire associer à chaque copie, une liste de booléens de longueur 20, indiquant pour chaque question, si la réponse donnée est la bonne. Le candidat Tom Matt a rendu la copie suivante pour l'épreuve 0 :

```
copTM = [4, 1, 5, 4, 3, 3, 1, 4, 5, 3, 5, 1, 5, 5, 5, 1, 3, 3, 3, 3].
```

La liste de booléens correspondante est alors :

```
corrTM = [True, False, False, True, True, False, False, False,
False, False, False, False, False, True, False, False, False, False,
True, True].
```

1. Écrire en Python une fonction `corrige` qui prend en paramètre `cop` et `corr`, deux listes d'entiers entre 1 et 5 et qui renvoie la liste des booléens associée à la copie `cop` selon la correction `corr`.

   Par exemple, `corrige(copTM, corr0)` renvoie `corrTM`.

La note attribuée à une copie est simplement le nombre de bonnes réponses. Si on dispose de la liste de booléens associée à une copie selon la correction, il suffit donc de compter le nombre de `True` dans la liste. Tom Matt obtient ainsi 6/20 à l'épreuve 0. On remarque que la construction de cette liste de booléens n'est pas nécessaire pour calculer la note d'une copie.

2. Écrire en Python une fonction `note` qui prend en paramètre `cop` et `corr`, deux listes d'entiers entre 1 et 5 et qui renvoie la note attribuée à la copie `cop` selon la correction `corr`, sans construire de liste auxiliaire.

   Par exemple, `note(copTM, corr0)` renvoie 6.

L'institut EN souhaite automatiser totalement la correction de ses copies. Pour cela, il a besoin d'une fonction pour corriger des paquets de plusieurs copies. Un paquet de copies est donné sous la forme d'un dictionnaire dont les clés sont les noms des candidats et les valeurs sont les listes représentant les copies de ces candidats. On peut considérer un paquet `p1` de copies où l'on retrouve la copie de Tom Matt :

```
p1 = {('Tom', 'Matt'): [4, 1, 5, 4, 3, 3, 1, 4, 5, 3, 5, 1, 5, 5, 5,
1, 3, 3, 3, 3], ('Lambert', 'Ginne'): [2, 4, 2, 2, 1, 2, 4, 2, 2, 5,
1, 2, 5, 5, 3, 1, 1, 1, 4, 4], ('Carl', 'Roth'): [5, 4, 4, 2, 1, 4,
5, 1, 5, 2, 2, 3, 2, 3, 3, 5, 2, 2, 3, 4], ('Kurt', 'Jett'): [2, 5,
5, 3, 4, 1, 5, 3, 2, 3, 1, 3, 4, 1, 3, 1, 3, 2, 4, 4], ('Ayet',
'Finzerb'): [4, 3, 5, 3, 2, 1, 2, 1, 2, 4, 5, 5, 1, 4, 1, 5, 4, 2,
3, 4]}.
```

3. Écrire en Python une fonction `notes_paquet` qui prend en paramètre un paquet de copies `p` et une correction `corr` et qui renvoie un dictionnaire dont les clés sont les noms des candidats du paquet `p` et les valeurs sont leurs notes selon la correction `corr`.

   Par exemple, `notes_paquet(p1, corr0)` renvoie `{('Tom', 'Matt'): 6, ('Lambert', 'Ginne'): 4, ('Carl', 'Roth'): 2, ('Kurt', 'Jett'): 4, ('Ayet', 'Finzerb'): 3}`.

   La fonction `notes_paquet` peut faire appel à la fonction `note` demandée en question 2, même si cette fonction n'a pas été écrite.

Pour éviter les problèmes d'identification des candidats qui porteraient les mêmes noms et prénoms, un employé de l'institut EN propose de prendre en compte les prénoms secondaires des candidats dans les clés des dictionnaires manipulés.

4. Expliquer si on peut utiliser des listes de noms plutôt qu'un couple comme clés du dictionnaire.

5. Proposer une autre solution pour éviter les problèmes d'identification des candidats portant les mêmes prénoms et noms. Cette proposition devra prendre en compte la sensibilité des données et être argumentée succinctement.

Un ingénieur de l'institut EN a démissionné en laissant une fonction Python énigmatique sur son poste. Le directeur est convaincu qu'elle sera très utile, mais encore faut-il comprendre à quoi elle sert.

Voici la fonction en question :

```
1  def enigme(notes) :
2      a = None
3      b = None
4      c = None
5      d = {}
6      for nom in notes :
7          tmp = c
8          if a == None or notes[nom] > a[1] :
9             c = b
10            b = a
11            a = (nom, notes[nom])
12         elif b == None or notes[nom] > b[1] :
13            c = b
14            b = (nom, notes[nom])
15         elif c == None or notes[nom] > c[1] :
16            c = (nom, notes[nom])
17         else :
18             d[nom] = notes[nom]
19         if tmp != c and tmp != None :
20             d[tmp[0]] = tmp[1]
21     return (a, b, c, d)
```

6. Calculer ce que renvoie la fonction `enigme` pour le dictionnaire `{('Tom', 'Matt'): 6, ('Lambert', 'Ginne'): 4, ('Carl', 'Roth'): 2, ('Kurt', 'Jett'): 4, ('Ayet', 'Finzerb'): 3}`.

7. En déduire ce que calcule la fonction `enigme` lorsqu'on l'applique à un dictionnaire dont les clés sont les noms des candidats et les valeurs sont leurs notes.

8. Expliquer ce que la fonction `enigme` renvoie s'il y a strictement moins de 3 entrées dans le dictionnaire passées en paramètre.

9. Écrire en Python une fonction `classement` prenant en paramètre un dictionnaire dont les clés sont les noms des candidats et les valeurs sont leurs notes et qui, en utilisant la fonction `enigme`, renvoie la liste des couples ((prénom, nom), note) des candidats classés par notes décroissantes.

   Par exemple, `classement({('Tom', 'Matt'): 6, ('Lambert', 'Ginne'): 4, ('Carl', 'Roth'): 2, ('Kurt', 'Jett'): 4, ('Ayet', 'Finzerb'): 3})` renvoie `[(('Tom', 'Matt'), 6), (('Lambert', 'Ginne'), 4), (('Kurt', 'Jett'), 4), (('Ayet', 'Finzerb'), 3), (('Carl', 'Roth'), 2)]`.

Le professeur Paul Tager a élaboré une évaluation particulièrement innovante de son côté. Toutes les questions dépendent des précédentes. Il est donc assuré que dès qu'un candidat s'est trompé à une question, alors toutes les réponses suivantes sont

également fausses. M. Tager a malheureusement égaré ses notes, mais il a gardé les listes de booléens associées. Grâce à la forme particulière de son évaluation, on sait que ces listes sont de la forme

`[True, True, …, True, False, False, …, False].`

Pour recalculer ses notes, il a écrit les deux fonctions Python suivantes (dont la seconde est incomplète) :

```
1  def renote_express(copcorr) :
2      c = 0
3      while copcorr[c] :
4          c = c + 1
5      return c
```

```
1  def renote_express2(copcorr) :
2      gauche = 0
3      droite = len(copcorr)
4      while droite - gauche > 1 :
5          milieu = (gauche + droite)//2
6          if copcorr[milieu] :
7              ...
8          else :
9              ...
10      if copcorr[gauche] :
11         return ...
12     else :
13         return ...
```

10. Compléter le code de la fonction Python `renote_express2` pour qu'elle calcule la même chose que `renote_express`.

11. Déterminer les coûts en temps de `renote_express` et `renote_express2` en fonction de la longueur n de la liste de booléens passée en paramètre.

12. Expliquer comment adapter `renote_express2` pour obtenir une fonction qui corrige très rapidement une copie pour les futures évaluations de M. Tager s'il garde la même spécificité pour ses énoncés. Cette fonction ne devra pas construire la liste de booléens correspondant à la copie corrigée, mais directement calculer la note.